

ECE 157B/272B - Homework 3

ECE 157/272 TAs

Winter 2025

Due: Sunday, March 2nd, 2025 by 11:59 PM
Start early!

Introduction

In the previous homework, we experienced a few purely text-based tasks, such as sentence completion and question-answering. In this homework, we will try out a text-to-image network called Stable Diffusion, developed by the CompVis group at LMU Munich (here)[1], Stability AI, and Runway ML. Stable Diffusion is a latent diffusion model, a kind of deep generative neural network that can generate detailed images conditioned on text descriptions. It can also perform other tasks such as inpainting, outpainting, and generating image-to-image translation guided by a text prompt.

In 2022, Google Research and Boston University collaborated to develop a fine-tuning technique for stable diffusion models, called DreamBooth [2]. This technique enables a specific subject to be incorporated into the stable diffusion model while still retaining high fidelity to its key visual characteristics in a new context (see Figure 1). In other words, this technique allows the trained model to create images of a subject that closely resemble the original training samples in various contexts.



Figure 1: An example figure from DreamBooth paper showing the fidelity and the new context

In this homework, we will be using the stable diffusion model for text-to-image generation and utilizing the DreamBooth technique to teach the model a new subject. We will then use the fine-tuned model to generate images of the new subject.

Use Cases

Image generation technology has many uses. We have listed a few below:

- Content creation: Image generation technology can be used to create new and unique images for digital media, such as social media posts, websites, and advertising.
- Design and fashion: Image-to-image technology can be used to generate new clothing designs, color schemes, and textures, allowing designers to create new products and variations more quickly and easily.
- Gaming: Image generation technology can be used to create realistic game environments and characters, making gameplay more immersive and engaging.
- Personalized Experience (DreamBooth): Creating a model dedicated to yourself. Having a model generate you in a different setting, clothing, hair color, hairstyle, a drawing/animation of yourself, etc. This allows you to explore yourself or just simply have a laugh.

Dataset

In this homework, you will be collecting your own dataset for training. Details will be discussed in the DreamBooth section.

Tasks

In this homework, we want you to get some hands-on experience with using stable diffusion models for text-to-image generation and using the DreamBooth technique for fine-tuning the model to a subject. The two tasks for this homework are:

1. Experiment with text-to-image generation with a stable diffusion model.
2. Use the DreamBooth technique to fine-tune a stable diffusion model for a person of your choice.

Stable Diffusion - Text-to-Image Generation

In this part of the homework, we will be using stable diffusion version 1.4, created by CompVis and made available from their hugging-face repository (here). You will use the given stable diffusion code to generate images of various class objects in various contexts. “Class” refers to a category of things (e.g., animal, dog, cat, human, backpack, buildings, etc.).

An example prompt might look like “Pikachu fine dining with a view to the Eiffel Tower” (see Figure 2). In this example, “Pikachu” is the class object, and “Eiffel Tower” is the context. If we change the context of the background, it will then become “Pikachu fine dining with a view to the Disney Castle”.

You can improve the quality of the images by using the Microsoft/Promptist model from hugging-face (here) to help you generate more descriptive prompts. Specifically, the Promptist model includes phrases in the prompt to generate images that fit certain styles. An example of the original prompt and the Promptist-refined prompt is shown below:

- Original: “A drawing of a person in the forest” (see Figure 3a)
- Promptist: “A drawing of a person in the forest, digital Art, Greg rutkowski, trending on artstation” (see Figure 3b)

In this example, Promptist added the styles “digital Art”, “Greg rutkowski”, and “trending on artstation,” which makes the output image a lot more pleasing.



(a) Pikachu fine dining with a view to the Eiffel Tower

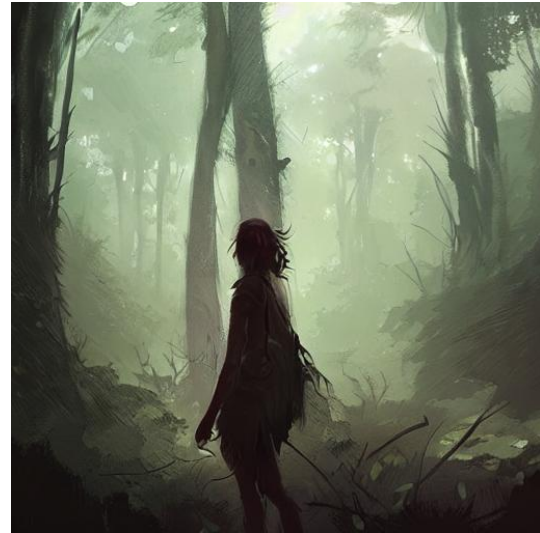


(b) Pikachu fine dining with a view to the Disney Castle

Figure 2: CompVis/Stable-Diffusion-1-4 generated images of Pikachu in different contexts



(a) A drawing of a person in the forest



(b) A drawing of a person in the forest, digital Art, Greg rutkowski, trending on artstation

Figure 3: Comparing original prompt and Promptist generated prompt for CompVis/Stable-Diffusion-1-4 generated images

For this task, submit the following:

- Submit 6 generated images along with their prompts. The prompts can be anything! Let your imagination run wild!
- Submit another 6 generated images along with their prompts, but this time use the Promptist model to refine the 6 previously used prompts.

DreamBooth - Adding a Person to the Model

In this part of the homework, we will be using the DreamBooth technique to fine-tune stable diffusion version 1.4 created by CompVis. The fine-tuning objective is to add a person to the model. Then, use the fine-tuned model to generate that person in various contexts. The contexts can be the person's appearance (e.g., hairstyle, hair color, clothing), the background (e.g., red carpet, supermarket, Disneyland), art style (e.g., realist portrait, concept art, animation), etc...

The DreamBooth steps in this homework are based on the hugging-face tutorial and the HuggingFace repository examples. They are somewhat different from the original paper. This is because we are using a different model than the original paper, and these are the settings your TAs found to be the best for their experiment settings in this part of the homework. Feel free to modify any parameters that you think might be more suitable for your experiment. Here are the steps to use DreamBooth:

1. Decide on a person you want to fine-tune your model for (yourself, a friend, a celebrity, etc...). **Check to see if that person is already recognized by the model by passing in a prompt to the model with their name in it.**
2. Collect images of the person. Then, crop and resize them to 512x512 (for CompVis/Stable-Diffusion-1-4). For better results, ensure the images are high quality, and collect at least 20 images.
3. Follow the given Google Colab notebook to install any libraries, log in to HuggingFace, and mount your Google Drive to the Google Colab workspace.
4. Creating necessary directories for storing your training images and model outputs. You will need to create the following directories:
 - `instance_dir` - for storing images of the person you choose. Please upload the images of the person you chose to this directory. It is recommended to create this in your Google Colab's working directory.
 - `class_dir` - for storing images of the *person* class. These images could be generated by running the given code, but considering the time it takes to generate the class images, we have given you the set of images we used in our experiment. Please unzip the given `person.class.images.zip` file and upload the *person* class images to this directory. It is recommended to create this directory in your Google Colab's working directory. We try to use Google Drive, but sometimes the script does not detect the images and causes a long runtime waiting for the model to generate the *person* class images.
 - `output_dir` - for storing output models and checkpoints during training. You just need to create the directory. It is recommended to create this in your Google Drive in order to preserve the model checkpoints in case your runtime times out.
 - NOTE: Your UCSB Google Account comes with 150GB of storage, of which this experiment uses around 50GB. Be careful not to use a regular Google account, which has less than 15GB of storage.
5. Generating the training command.
 - You shouldn't change the `model_name` unless you think another model is a better fit for this task. If your person is an anime character, then you will want to use a model trained on anime characters instead.
 - Fill in the `instance_dir`, `class_dir`, and `output_dir` with the paths of the directories you just created.
 - Fill in the `instance_prompt` using the following structure: "A `sks` person". Here "`sks`" is the token representing that person's name, and "person" is the class (it gives a hint to your model that you are trying to fine-tune for a person instance). *In the future, if you are going to fine-tune the model for different class, let say a dog, then you will replace "person" with "dog".*

- Fill in the `class_prompt` using the following structure: “A person”. Same as previous bullet point, “person” is the class of interest. The `class_prompt` is used to generate images to be used during training to avoid the model from overfitting (though it will still happen, but at a lesser degree). It does so by generating a set of class images using the `class_prompt` and using them to preserve the original class concepts.
 - `num_class_images` indicates the number of class images to generate to preserve the class concept. We have given you 1500 images so you can fill in 1500 in this field. If you want to use more images, you can increase this number and have the script generate more images.
 - `max_training_steps` is the number of training steps. We found 500 to be sufficient in our experiment. You can adjust this as you see fit.
 - `checkpointing_steps` indicates the frequency of saving the model checkpoints. We set it to 100 meaning checkpoints will be saved for every 100 training steps.
 - `learning_rate` is set to 0.000002 as suggested by a HuggingFace tutorial (here). We also found this number to perform pretty well for fine-tuning a person.
 - Select `resume_checkpoint` and enter the `checkpoint_number` if you want to resume training from an earlier checkpoint. This will be useful when your model overfits and you want to return to an earlier checkpoint where your model hasn’t overfit and continue the training.
6. You can run the code block to generate the training code. Then, you will run the xterm window code block to start a xterm window where you will paste your training command to start the training.
- The training should take around an hour on Google Colab’s GPU.
 - If you see an estimated execution time over two hours, then the code probably did not find your class images and started generating images. In this case, verify you enter the complete `class_dir` path correctly.
 - To keep the Google Colab from timing out while the training is in progress, run the code block below the xterm window code block. Terminate this code block when training is complete or if you need to run other code blocks.
 - **NOTE: Terminating any code block during training will interrupt the xterm window and kill the training.**
7. Now, you have your trained model and the checkpoints along the way. Time to test them!
- Select the `scheduler_name` for the inference model. Then, run the cell to load the model with that scheduler. DDIM worked better for our experiment.
 - For the “Generate image from prompt” cell block, enter a prompt to test your model’s ability to generate the person in different context. Modify the `num_inference_steps` and `guidance_scale` as needed.
 - Increase the `num_inference_steps` if your images have a lot of noise.
 - Increase the `guidance_scale` if the image is not representative of the prompt.
 - To test whether the model is overfitting to the training samples, you can modify the prompt to change the context. For example, try to give the person a different hair color or outfit. If the image fails to show those contexts after a few image generations, then your model is most likely overfitting.
 - To fix the overfitting problem, you will need to load a model from an earlier checkpoint and repeat the verification process. If none of the checkpoints yield good results, you will need to retrain the model with different parameters.
 - If you have a checkpoint that is able to show context change, but it is not generating the face correctly, you can try to increase the `num_inference_steps` or modify the prompt using Promptist as a guidance. If all else fails, fine-tune the model again but use the `resume_checkpoint` options to start from a checkpoint rather than starting from scratch.

8. Lastly, remember to save the model to your hugging-face hub so you can use it later on.

For this task, submit the following items:

- The training images you collected for the person.
- Your training parameters.
- The checkpoint number that gave you the best model.
- The inference parameters (`scheduler_name`, `num_inference_steps`, and `guidance_scale`).
- 12 generated images of the person you trained on in various contexts (e.g. different hair color, outfit, background, art style, etc...) along with their respective prompts.

What to turn in

- For the report, use the given slide template to make *slides* that show the results/submissions requested in the above sections. Export and submit the slides as a PDF document to GradeScope.

References

- [1] Robin Rombach et al. “High-Resolution Image Synthesis with Latent Diffusion Models”. In: *CoRR* abs/2112.10752 (2021). arXiv: 2112.10752. URL: <https://arxiv.org/abs/2112.10752>.
- [2] Nataniel Ruiz et al. *DreamBooth: Fine Tuning Text-to-Image Diffusion Models for Subject-Driven Generation*. 2022. DOI: 10.48550/ARXIV.2208.12242. URL: <https://arxiv.org/abs/2208.12242>.